

Vhdl-parse

Documentation

Version 0.1.0, 2026-07-26

Sylvain Tertois

<https://github.com/daixiwen/typst-vhdl-parse>

Summary

Vhdl-parse is a typst module that parses VHDL files and extracts information from them that can be presented in a typst document. It makes writing documentation for VHDL code easier and lets use the VHDL code as single source of truth.

It only extracts information and provides it in structures. It is up to the user to format it in a way to present it in a readable manner.

Table of contents

1 Usage	2
1.1 Invocation	2
1.2 Comments	2
2 Functions reference	3
2.1 parse	3
2.2 port-list	4
2.3 generic-list	5
2.4 fsm	6
2.5 fsm-dot	8
2.6 instances-list	11
2.7 constants-list	12
2.8 signals-list	13
2.9 types-list	14
3 Implementation details	17
3.1 Architecture	17
3.2 Caching	17

1 Usage

1.1 Invocation

Import the `vhdl-parse` module as usual:

```
1 #import "@preview/vhdl-parse:0.1.0" typ
```

Use the `parse()` function to parse your VHDL file.

```
1 #let parsed-file = vhdl-parse.parse(  
2     "test.vhd",  
3     read("test.vhd")  
4  
5 #assert(parsed-file.messages.len() == 0, message. "errors were found in the VHDL file")
```

The object returned is used as parameter for all the other functions to extract different kind of information from the VHDL contents. The `message` field contains errors returned by the VHDL parser.

Please note that the `vhdl-parse` module does not present the extracted data in any ready-to-display state. It returns arrays and dictionary with the requested data, and it is up to the user to write a template to present the data in a pretty and consistent way across the documentation.

Please refer to section 2 for more details about all the functions that can be called with the parsed file.

1.2 Comments

When parsing VHDL, the module looks after comments to describe the objects it finds. It looks for both comments on the same line than the object, and for comments on the line before it, as shown in the following examples:

```
1 -- this is a leading comment (before the object) vhdl  
2 signal my_signal: std_logic;  
3  
4 signal my_other_signal: std_logic; -- this is a trailing comment (with the object)
```

If an object has both a leading and a trailing comment, the parser doesn't know which one to pick. This is why there is a `comment-priority` parameter, which tells it which comment to pick when both are detected. The `parse()` function has the `comment-priority` parameter to give the default value, but it can then be overridden in all other function calls if needed.

2 Functions reference

2.1 parse

Parses a VHDL file

The return value is a dictionary, used as parameter in the other functions to extract information from the parsed VHDL file. The `messages` element in the returned dictionary is a list of warnings or errors from the parser.

2.1.1 Example

```
1 #let parsed-file = vhdL-parse.parse( typ
2   "test.vhd",
3   read("test.vhd"))
4
5 Messages from the parser:
6
7 #if parsed-file.messages.len() == 0 [
8   no messages
9 ] else {
10   for message in parsed-file.messages [
11     - #message ]
12 }
```

Messages from the parser:

no messages

2.1.2 Parameters

```
parse(
  file-name: string,
  contents: string bytes,
  vhdL-variant: string int,
  comment-priority: string
) -> dictionary
```

file-name string

The VHDL file name

contents string or bytes

The VHDL code, usually the result of a `read` operation on the VHDL file

vhdL-variant string or int

(optional) The VHDL variant

Default: 2008

comment-priority string

(optional) The default comment priority. Either "leading" or "trailing", see section 1.2

Default: "trailing"

2.2 port-list

Returns the portlist from the first entity found in the parsed file

2.2.1 Return structure

an array of dictionaries, each item having the following elements:

name	type	description
name	string	the port name
mode	string	the port mode (in, out, inout, buffer)
port-type	string	the VHDL type of the port
constraint	string or none	the type constraint, for example: "(15 downto 0)"
description	string or none	a comment describing the port
default-value	string or none	the port default value

2.2.2 Example

```
1  #let ports = vhdl-parse.port-list(typ  
2    parsed-file)  
3  
4  Ports list:  
5  
6  #table(  
7    columns: (2cm, 1.5cm, 4cm, 6cm),  
8    table.header([name], [mode], [type],  
9      [description]),  
10    ..for entry in ports {  
11      ( [#entry.name],  
12        [#entry.mode],  
13        [{entry.port-type}  
14          #{entry.constraint}],  
15        [#entry.description])  
16    }  
17  )
```

Ports list:

name	mode	type	description
clock	in	std_logic	main clock
sreset	in	std_logic	main reset, synchronous, active high
output_a	out	std_logic	a regular output
input_b	in	std_logic	one input
output_c	out	std_logic	another output
input_d	in	std_logic	another input
data_in	in	std_logic_vector(15 downto 0)	
data_out	out	std_logic_vector(15 downto 0)	data out

2.2.3 Parameters

```
port-list(  
  parsed-file: dictionary ,  
  comment-priority: string or none  
) -> array
```

parsed-file dictionary

The parsed file object, as returned by `parse()`.

comment-priority string or none

(optional) override the default comment priority, either “leading” or “trailing”, see section 1.2

Default: none

2.3 generic-list

Returns the generics list from the first entity found in the parsed file

2.3.1 Return structure

an array of dictionaries, each item having the following elements:

name	type	description
name	string	the generic name
generic-type	string	the VHDL type of the generic
constraint	string or none	the type constraint, for example: "(15 downto 0)"
description	string or none	a comment describing the generic
default-value	string or none	the generic default value

2.3.2 Example

```
1 #let generics = vhdl-parse.generic-list( typ
2   parsed-file)
3
4   Generics list:
5
6   #table(
7     columns: (2cm, 4cm, 6cm, 2cm),
8     table.header([name], [type],
9                  [description], [default]),
10    ..for entry in generics {
11      ( [#entry.name],
12        [#entry.generic-type]
13        [#entry.constraint]},
14        [#entry.description],
15        [#entry.default-value])
16    }
```

Generics list:

name	type	description	default
flag	boolean	a flag, can be true or false	
value	std_logic_vector(15 downto 0)	a value with a default	x"DEAD"

2.3.3 Parameters

```
generic-list(
  parsed-file: dictionary,
  comment-priority: string or none
) -> array
```

parsed-file dictionary

The parsed file object, as returned by `parse()`.

comment-priority string or none

(optional) override the default comment priority, either "leading" or "trailing", see section 1.2

Default: none

2.4 fsm

Returns information about a state machine found in the parsed file

This function looks for a case statement and assignments within the different cases:

```
1  case my_state is
2      when state_1 =>
3          next_state <= state_2
4
5      when state_2 =>
6          next_state <= state_1
7
8      when others =>
9          next_state <= state_1
10 end case;
```

vhdl

Both signal and variable assignments are detected. The signal or variable that is holding the current state (`my_state` in the example above) is defined with the `read-variable-name` parameter in the function call, while the signal or variable assigned to the next state (`next_state` in the example above) is defined with the `write-variable-name` parameter. If the same signal is both read and written, the name just needs to be defined in `read-variable-name` and `write-variable-name` can be kept to `none` (the default). If an assignment is found in the `when others` clause, the destination of the assignment will be stored as the default state.

`fsm()` will attempt to find a description for each transition. For that it will look for a comment near the **condition** (an `if`, `elsif` or `else` statement) before the assignment. If it doesn't find a comment, it will attempt to use the test condition itself (inside the `if` or `elsif`) as description.

2.4.1 Return structure

a dictionary with the following elements:

name	type	description
default-state	string	the default state (i.e. found in an "others" case)
states	array	an array of states

The `states` array contains dictionaries, with each item having the following elements:

name	type	description
name	string	the state name
description	string or none	the comment describing the state
transitions	array	an array of transitions

The `transitions` arrays contain dictionaries, with each item having the following elements:

name	type	description
destination	string	the state it is transitioned to
condition	string	the condition for the transition, if found or an empty string
description	string or none	the comment describing the condition, if found

2.4.2 Example

```

1  #let fsm = vhdl-parse.fsm(parsed-
   file,"fsm_state")
2
3  List of states:
4
5  #table(
6      columns: (3cm, 7cm),
7      table.header([state], [description]),
8      ..for state in fsm.states {
9          ( [#state.name], [#state.description])
10     }
11 )

```

List of states:

state	description
reset	start and reset state
idle	normal state when nothing happens
read_input	waiting for input
write_output	send output

2.4.3 Parameters

```

fsm(
  parsed-file: dictionary ,
  read-variable-name: string ,
  write-variable-name: string none ,
  comment-priority: string none
) -> dictionary

```

parsed-file dictionary

The parsed file object, as returned by `parse()`.

read-variable-name string

The name of the signal or variable holding the current fsm state

write-variable-name string or none

(optional) The name of the signal or variable holding the next fsm state, if different from read-variable-name

Default: none

comment-priority string or none

(optional) override the default comment priority, either “leading” or “trailing”, see section 1.2

Default: none

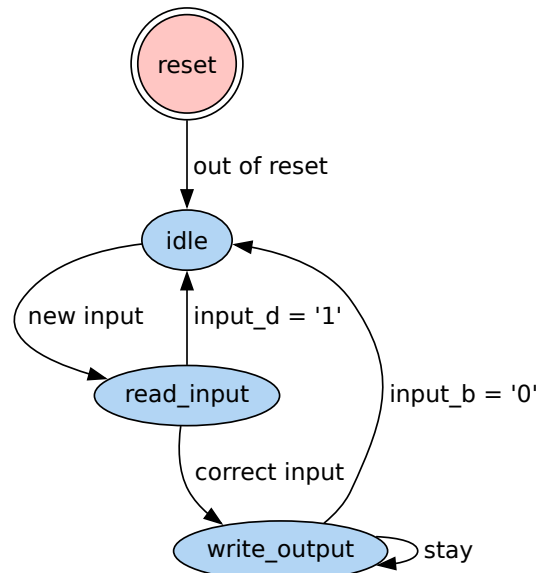
2.5 fsm-dot

Returns a description of an FSM in the DOT format, ready to be drawn by the `diagraph` package.

For more information about FSM detection please refer to the [fsm\(.\)](#) function.

2.5.1 Example

```
1 #let dot = vhdL-parse.fsm-dot(  
2   parsed-file,  
3   "fsm_state",  
4   font-name: "DejaVu Sans,  
5     Helvetica,Arial,sans-serif",  
6   state-background-color: blue.lighten(70%),  
7   default-state-background-color:  
8     red.lighten(70%)  
9 )  
10 #import "@preview/diagraph:0.3.7"  
11 #diagraph.render(dot)
```



2.5.2 Parameters

```
fsm-dot(  
  parsed-file: dictionary ,  
  read-variable-name: string ,  
  write-variable-name: string none ,  
  comment-priority: string none ,  
  left-to-right: bool ,  
  font-name: string ,  
  state-shape: string ,  
  state-background-color: color ,  
  state-line-color: color ,  
  state-text-color: color ,  
  state-font-size: float ,  
  default-state-shape: string ,  
  default-state-background-color: color none ,  
  default-state-line-color: color none ,  
  default-state-text-color: color none ,  
  default-state-font-size: float none ,  
  transition-line-color: color none ,  
  transition-text-color: color none ,  
  transition-font-size: float none  
) -> string
```

parsed-file dictionary

The parsed file object, as returned by [parse\(.\)](#)

read-variable-name `string`

The name of the signal or variable holding the current fsm state

write-variable-name `string` or `none`

(optional) The name of the signal or variable holding the next fsm state, if different from `read-variable-name`

Default: `none`

comment-priority `string` or `none`

(optional) override the default comment priority, either “leading” or “trailing”, see section 1.2

Default: `none`

left-to-right `bool`

(optional) if true, make a left-to-right diagram instead of top-to-down

Default: `false`

font-name `string`

(optional) the font to use (has to be accessible to Typst)

Default: `"Helvetica,Arial,sans-serif"`

state-shape `string`

(optional) the shape to use for states (refer to the Graphviz documentation for a list)

Default: `"ellipse"`

state-background-color `color`

(optional) the background color for states

Default: `white`

state-line-color `color`

(optional) the line color for states

Default: `black`

state-text-color color

(optional) the color of text for states

Default: black

state-font-size float

(optional) the text size for states (in points)

Default: 14

default-state-shape string

(optional) the shape to use for the default state (refer to the Graphviz documentation for a list)

Default: "doublecircle"

default-state-background-color color or none

(optional) the background color for the default state. If none, use the same one as for the regular states

Default: none

default-state-line-color color or none

(optional) the line color for the default state. If none, use the same one as for the regular states

Default: none

default-state-text-color color or none

(optional) the color of text for the default state. If none, use the same one as for the regular states

Default: none

default-state-font-size float or none

(optional) the text size for the default state (in points). If none, use the same one as for the regular states

Default: none

transition-line-color color or none

(optional) the line color for the transitions. If none, use the same one as for the regular states

Default: none

transition-text-color color or none

(optional) the color of text for the transitions. If none, use the same one as for the regular states

Default: none

transition-font-size float or none

(optional) the text size for the transitions (in points). If none, use the same one as for the regular states

Default: none

2.6 instances-list

Returns the instances list from the first entity found in the parsed file

2.6.1 Return structure

an array of dictionaries, each item having the following elements:

name	type	description
label	string	the instantiation label
entity	string	the entity being instantiated
description	string or none	a comment describing the instance
generics-map	array	an array of generics
ports-map	array	an array of ports

generics-map and ports-map are arrays of dictionaries, each item having the following elements:

name	type	description
origin	string	name of the generic or port on the entity side
expression	string	the expression assigned to the generic or port

2.6.2 Example

```
1 List of Instances
```

```
typ
```

```

2
3 #let instances = vhdl-parse.instances-
  list(parsed-file)
4
5 #table(
6     columns: (3cm, 3cm, 7cm),
7     table.header([label], [entity],
8                   [description]),
9     ..for element in instances {
10         ( [#element.label], [#element.entity],
11           [#element.description])
12     }

```

List of Instances

label	entity	description
my_instance	work.comp	an example instantiation

2.6.3 Parameters

```

instances-list(
    parsed-file: dictionary ,
    comment-priority: string none
) -> array

```

parsed-file dictionary

The parsed file object, as returned by `parse()`.

comment-priority string or none

(optional) override the default comment priority, either “leading” or “trailing”, see section 1.2

Default: none

2.7 constants-list

Returns the constants list from the architecture or package definition in the parsed file

2.7.1 Return structure

an array of dictionaries, each item having the following elements:

name	type	description
name	string	the constant name
object-type	string	the constant type
constraint	string or none	the type constraint (x downto y)
expression	string or none	the constant value
description	string or none	a comment describing the constant

2.7.2 Example

```
1 >>> #set text(font: ("DejaVu Sans", "Arial", "Helvetica"))
2 List of Constants
3
4 #let constants = vhdL-parse.constants-list(parsed-file)
5
6 #table(
7     columns: (2cm, 4cm, 3cm, 4cm),
8     table.header([name], [type], [description], [value]),
9     ..for entry in constants {
10         ( [#entry.name],
11           [#entry.object_type]#entry.constraint},
12           [#entry.description],
13           [#entry.expression])
14     }
15 )
```

Example

2.7.3 Parameters

```
constants-list(
  parsed-file: dictionary ,
  comment-priority: string none
) -> array
```

parsed-file dictionary

The parsed file object, as returned by [parse\(\)](#).

comment-priority string or none

(optional) override the default comment priority, either “leading” or “trailing”, see section 1.2

Default: none

2.8 signals-list

Returns the signals list from the architecture or package declaration in the parsed file

2.8.1 Return structure

an array of dictionaries, each item having the following elements:

name	type	description
name	string	the signal name
object-type	string	the signal type
expression	string or none	the signal initial value

name	type	description
constraint	string or none	the type constraint (x downto y)
description	string or none	a comment describing the signal

2.8.2 Example

```

1  List of Signals
2
3  #let signals = vhdl-parse.signals-list(parsed-
4    file)
5
6  #table(
7    columns: (2cm, 3.5cm, 5cm, 3.5cm),
8    table.header([name], [type], [description],
9      [value]),
10   ..for entry in signals {
11     ( [#entry.name],
12       [#entry.object_type]
13       [#entry.constraint]),
14     [#entry.description],
15     [#entry.expression])
16   }
17 )

```

List of Signals

name	type	description	value
fsm	fsm_wrapper_t	state machine signal	
mysignal	unsigned(15 downto 0)	a signal with a comment on the same line	(others => '0')

2.8.3 Parameters

```

signals-list(
  parsed-file: dictionary,
  comment-priority: string or none
) -> array

```

parsed-file dictionary

The parsed file object, as returned by `parse()`.

comment-priority string or none

(optional) override the default comment priority, either “leading” or “trailing”, see section 1.2

Default: none

2.9 types-list

Returns the types list from the architecture or package declaration in the parsed file

2.9.1 Return structure

an array of dictionaries, each item having the following elements:

name	type	description
name	string	the type name
kind	string	the type kind: "enumeration", "record", "subtype" or "array"
description	string or none	a comment describing the type
definition	see below	an object with the type definition. The structure depends on the type kind

The definition object is different depending on the type kind:

- Enumeration: an array of dictionaries, each item having the following elements:

name	type	description
element-name	string	the enumeration element name
description	string or none	a comment describing the element

- Record: an array of dictionaries, each item having the following elements:

name	type	description
element-name	string	the record element name
element-type	string	the record element type
description	string or none	a comment describing the record element

- Subtype: a dictionary with the following elements:

name	type	description
subtype	string	the type name
constraint	string or none	the constraint (x downto y)

- Array: a dictionary with the following elements:

name	type	description
range	string	the array range(s)
subtype	string	the array element type

2.9.2 Example

```

1 List of types typ
2
3 #let types = vhdl-parse.types-list(parsed-
4   file)
5 #table(
6   columns: (3.5cm, 3cm, 6cm),
7   table.header([name], [kind],
8     [description]),
9   ..for entry in types {
10     ( [#entry.name], [#entry.kind],
11       [#entry.description] )
12   }
13 )

```

List of types

name	kind	description
fsm_t	enumeration	enumeration type for the FSM
fsm_wrapper_t	record	a wrapper around the fsm type to test record element access in FSM detection
byte_t	subtype	a subtype
matrix_t	array	an array

2.9.3 Parameters

```
types-list(  
  parsed-file: dictionary ,  
  comment-priority: string none  
) -> array
```

parsed-file dictionary

The parsed file object, as returned by `parse(.)`.

comment-priority string or none

(optional) override the default comment priority, either “leading” or “trailing”, see section 1.2

Default: none

3 Implementation details

This section is only relevant if you are interested in the inner workings of the package or if you want to make any changes.

3.1 Architecture

The package is made of a lightweight Typst file calling a WASM plugin. the plugin itself is written in Rust and uses the [vhdl_lang](#) crate. The plugin and the Typst file communicate parameters and return values using byte arrays, and the CBOR protocol is used on both sides to transfer more complex structures.

There is a very basic `AGENTS.md` file in the repository. I used [OpenCode](#) to kickstart the project because I couldn't find how to use the [vhdl_lang](#) crate by myself, and to write some tests. The rest was written manually, because it is pretty basic code, walking through the structures returned by [vhdl_lang](#) to find the relevant information and putting it in structures which can be serialized for Typst. I found it easier to just write the code than to try to det en LLM to put it exactly in the way I wanted to. Maybe I'm just bad at prompting.

3.2 Caching

The plugin caches results from a VHDL parsing, because I thought it would be very inneficient to redo it at each function call. Unfortunately it makes it difficult to make the plugin [pure](#) as required by Typst: a plugin function call must not have any observable side effects on future plugin calls, and given the same arguments, it must always return the same value.

I have therefore implemented a cache system. The [parse\(\)](#) function stores the results of the parsing in a static cache, using the [lazystatic](#). It returns an ID for typst, and the other functions use that ID as a parameter, so that the parsed VHDL can be extractd again from the cache. To respect the purity requirement, the other functions also send the VHDL file name and the contents with the ID. That way the plugin can parse the VHDL file again if needed, and technically the function is not dependent on previous calls, but still benefits from a cache.